Zewail City of Science, Technology and Innovation
University of Science and Technology
School of Computational Sciences and Artificial Intelligence

CSAI 203 - Fall 2026
Introduction to Software Engineering

Online Quiz & Auto-Grading System

Software Requirements Specification - Phase 2

Team Number: 1

Team Member	University ID	University Email
Abdelrahman Sabry	202612548	s-a.mehena48@zewailcity.edu.eg
Mohamed Ismael	202402049	s-mohamed.ismael@zewailcity.edu.eg
Saeed Mohamed	202400436	s-saeid.msaeid@zewailcity.edu.eg
Loai Mohamed	202400433	s-loai.bjnordi@zewailcity.edu.eg

Representative Contact: Saeed Mohamed - s-saeid.msaeid@zewailcity.edu.eg

23 October 2026

Team review copy

Table of Contents

Phase 2 Software Requirements Specification

1. Introduction	3
1.1 Purpose; 1.2 Scope	3
1.3 Definitions, Acronyms, and Abbreviations	3
1.4 References; 1.5 Overview	3
2. Overall Description	4
2.1 Product Perspective through 2.8 Assumptions and Dependencies	4
3. Specific Requirements	5
3.1 Functional Requirements (FR-01 to FR-20)	5-8
3.2 Use Case Model	9
3.2.1 Use Case Diagrams	9-10
3.2.2 Use Case Descriptions (UC-01 to UC-12)	11-13
3.3 Domain Model	14-15
3.3.1 Conceptual Class Diagram	14
3.3.2 Class Descriptions	15
3.4 Non-Functional Requirements	16
3.5 External Interface Requirements	17
4. Appendices	15, 17
4.1 Appendix A: Data Dictionary	15
4.2 Appendix B: Glossary	17

Page count: 17 pages, including the cover and this contents page. Body text uses 12-point Times with 1.5 line spacing.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the externally observable behavior and quality targets of the Online Quiz & Auto-Grading System. Students, instructors, administrators, and the project team will use it as the shared basis for design, implementation, testing, and evaluation. Each requirement is identified so later diagrams, tests, and implementation work can be traced back to an agreed expectation.

1.2 Scope

The system is a browser-based assessment tool for a university course. Instructors create question banks and timed quizzes, assign them to courses, and review performance. Students access eligible quizzes, submit attempts, and receive results according to the instructor's release policy. The system automatically marks objective questions with configured answer keys and supports instructor marking for written responses. This phase defines requirements only; it does not specify a final technology stack, implement an LMS integration, or use a machine-learning model to judge free-text answers.

1.3 Definitions, Acronyms, and Abbreviations

Attempt: one student's recorded session for a quiz. **Objective item:** a question with a deterministic key, such as multiple choice or true/false. **Subjective item:** a response requiring instructor judgment. **RBAC:** role-based access control. **SRS:** Software Requirements Specification. **NFR:** non-functional requirement. Further terms appear in Appendix B.

1.4 References

[1] Dr. Mohamed Sami Rakha, *CSAI 203 Course Project, Fall 2026*, course project brief supplied to the team, 2026.

[2] CSAI 203 lecture slides, *UML Use Case IEEE Description Template*, Fall 2026, as identified in the course project brief.

1.5 Overview

Section 2 describes the product context, users, constraints, and assumptions. Section 3 defines twenty functional requirements, the use case model, the conceptual domain model, five measurable quality requirements, and external interfaces. Section 4 provides a concise data dictionary and glossary. The document is limited to Phase 2 requirements.

2. Overall Description

2.1 Product Perspective

The product is a standalone web application for course assessments, storing course, quiz, question, attempt, and result records in its own database. It is not an LMS plug-in and does not require campus single sign-on in this phase; project accounts are provisioned locally.

2.2 Product Functions

The system manages accounts, roles, courses, rosters, question banks, quiz schedules, timed attempts, saved answers, objective and manual grading, result release, and performance summaries.

2.3 User Classes and Characteristics

Students take assigned quizzes and review released grades. **Instructors** manage courses, questions, quizzes, grading, and reports. **Administrators** provision accounts and roles. All users need basic browser skills; programming experience is not required.

2.4 Software Stakeholders

Students need fair timing, saved answers, and clear results. Instructors need accurate grading and reports. The project team needs traceable requirements and repeatable tests. Course staff and the university need protected account and assessment data.

2.5 Operating Environment

The application uses a hosted web server and relational database. Users access it from a current desktop or mobile browser over the internet. Quiz deadlines use server time, not the user device clock.

2.6 Design and Implementation Constraints

The implementation must use course-approved front-end and back-end technologies and the required GitHub repository structure. This SRS is framework-neutral because no stack is specified in the brief. Objective grading is deterministic; written grading remains instructor-controlled.

2.7 User Documentation

The project will provide a student FAQ, an instructor guide for course and quiz management, grading and reports, and a concise administrator guide for account and role setup.

2.8 Assumptions and Dependencies

Users have a supported browser and network access; instructors provide accurate rosters, keys, rubrics, schedules, and point values. Hosting and database services are available during quiz windows. Email, campus single sign-on, proctoring, payment, and LMS integration are out of scope.

3. Specific Requirements

3.1 Functional Requirements

FR-01 - Sign in and sign out

The system shall allow an active student, instructor, or administrator to sign in with a registered university email and password. Valid credentials start a session and open the user's permitted home page. Invalid credentials return a generic error without revealing whether the email exists; inactive accounts are denied. A user can sign out, and an idle session expires after the configured inactivity interval.

FR-02 - Enforce role permissions

The system shall check the authenticated user's role and course relationship before every protected action. Students may access only their own attempts and released results; instructors may manage only assigned courses and their assessments; administrators may provision accounts and roles. Directly opening another record by changing its URL or identifier shall be denied and logged without exposing its data.

FR-03 - Manage courses

An authorized instructor or administrator shall create, view, edit, archive, and reopen a course using its code, title, term, and status. The system shall reject a duplicate course code in the same term and require a title and term before saving. Archiving a course shall prevent new quiz attempts while retaining historical attempts and grades for authorized review.

FR-04 - Manage course enrollments

An instructor or administrator shall add and remove students from a course roster using registered university accounts. The system shall prevent duplicate active enrollment and shall show a clear error for an unknown account. Removing an enrollment blocks access to future quizzes but preserves past attempts and grades. Roster changes shall be recorded with actor and timestamp.

FR-05 - Create and maintain questions

An instructor shall create, view, edit, and archive question-bank items with a prompt, question type, point value, and any required options, answer key, or rubric. The system shall reject missing required fields, non-positive points, and invalid option sets. If a question has already been used in a published quiz, changing it shall create a new version or require a safe confirmation so past attempts keep their original wording and key.

3.1 Functional Requirements (continued)

FR-06 to FR-10

FR-06 - Support objective and written question types

The system shall support single-choice, multiple-select, true/false, and instructor-marked written-response questions. For objective items, the instructor shall define the accepted answer or exact answer set. For written items, the instructor shall supply a maximum score and optional rubric. The editor shall display which fields are required for each type and shall not allow an item to be published with an incomplete key or rubric.

FR-07 - Organize and find bank questions

An instructor shall be able to search and filter the course question bank by prompt text, question type, topic tag, and status. Search results shall show enough information to distinguish questions without exposing answer keys to students. If no item matches, the system shall display an empty-state message and keep the search criteria available for adjustment.

FR-08 - Compose a quiz

An instructor shall create a quiz with a title and instructions, then add, remove, reorder, and assign points to questions from an authorized course bank. The system shall show the total possible score and require at least one active question before saving. A question may appear only once in a quiz unless the instructor explicitly confirms a repeated item, and the preview shall reflect the current order and points.

FR-09 - Configure quiz availability and limits

An instructor shall set the opening and closing time, duration, attempt limit, and result-release policy for a quiz. The system shall validate that the close time follows the open time, duration and attempt limit are positive, and the settings are saved in server time. Invalid combinations shall be rejected with field-level explanations; changing a published window shall show the instructor the effect on active attempts.

FR-10 - Randomize eligible quiz content

When enabled by the instructor, the system shall shuffle question order and/or the order of choices for each new attempt, while preserving each question's correct answer mapping and point value. Questions marked as fixed shall remain in their specified position. The same saved attempt shall retain its original order when resumed; refreshing a page shall not reshuffle or alter a student's assessment.

3.1 Functional Requirements (continued)

FR-11 to FR-15

FR-11 - Show eligible quizzes to students

A student shall see quizzes assigned to courses in which the student has an active enrollment. Each entry shall show the course, opening and closing time, duration, attempt count, and current status (upcoming, open, submitted, or closed). A student shall not see answer keys, private instructor notes, or quizzes belonging to another course. When no quiz is eligible, the system shall explain that none are currently available.

FR-12 - Start an authorized attempt

When a student starts a quiz, the system shall verify active enrollment, the server-side availability window, remaining attempts, and that no conflicting active attempt exists. If eligible, it creates one attempt with a server-recorded start time and deadline and presents the quiz instructions. If any check fails, no attempt is created and the student receives a reason that does not reveal restricted course data.

FR-13 - Enter and navigate answers

During an active attempt, a student shall be able to enter or change responses, move between questions, and see which items are answered or unanswered. The interface shall distinguish saved from pending responses and shall never expose the answer key. If a response is malformed or exceeds the configured input limit, the system shall identify the affected item and retain other valid responses.

FR-14 - Save and recover an attempt

The system shall save a response after an explicit save action and at configured autosave intervals, recording the latest accepted server timestamp. A student who loses connectivity may return to the same open attempt and see the last server-confirmed answers and remaining server-calculated time. The interface shall not label an unsent answer as saved; a conflict shall preserve the last accepted value and tell the student to retry.

FR-15 - Submit or time out an attempt

A student shall be able to submit an active attempt and receive a confirmation of successful receipt. The system shall permit a final review before confirmation, clearly mark unanswered questions, and make submission idempotent so repeated clicks do not create duplicate attempts. When the server deadline is reached, the system shall close editing and submit the last saved responses automatically; a late network request shall not extend the deadline.

3.1 Functional Requirements (continued)

FR-16 to FR-20

FR-16 - Automatically grade objective responses

After submission, the system shall compare each objective response with the answer key version used to publish the quiz, apply its configured scoring rule, and calculate an objective subtotal. Unanswered questions receive the configured default score, normally zero. A multiple-select response earns credit only when it matches the accepted set unless partial credit was explicitly configured. The system shall store per-item results and the grading rule version for review.

FR-17 - Manually grade written responses

An instructor shall be able to review each written response, assign a score from zero through the configured maximum, and optionally provide student-visible feedback. The system shall reject scores outside that range, preserve unsaved work as pending, and record the grader and timestamp after save. When a saved mark changes, the system shall recalculate the attempt total and keep an audit record of the prior and new values.

FR-18 - Release grades and feedback

An instructor shall release results according to the quiz policy, either for all eligible attempts or for a selected group. The system shall not expose a result before release and shall prevent release while required manual marks are incomplete unless the instructor explicitly chooses a partial-release policy. The release action shall record actor and time. Students shall be notified within the application when a result becomes available.

FR-19 - Review personal results

After release, a student shall see the attempt score, question-level marks, and feedback allowed by the instructor's policy. The system shall hide correct answers and instructor-only notes when those are not released. The result page shall identify unanswered items and distinguish objective from manually marked items. A student who has no released result shall receive a clear status message instead of an empty or misleading score.

FR-20 - View and export performance summaries

An authorized instructor shall filter a quiz report by course, quiz, and attempt status and view counts, score distribution, average, and per-question correctness. The system shall present a clear empty state when no attempts exist. The instructor may export the currently filtered summary as CSV; the exported values shall match the displayed filters and shall exclude passwords, answer keys, and data from courses the instructor cannot access.

3.2 Use Case Model

3.2.1 Use Case Diagrams

Figure 1. Instructor setup and quiz publication

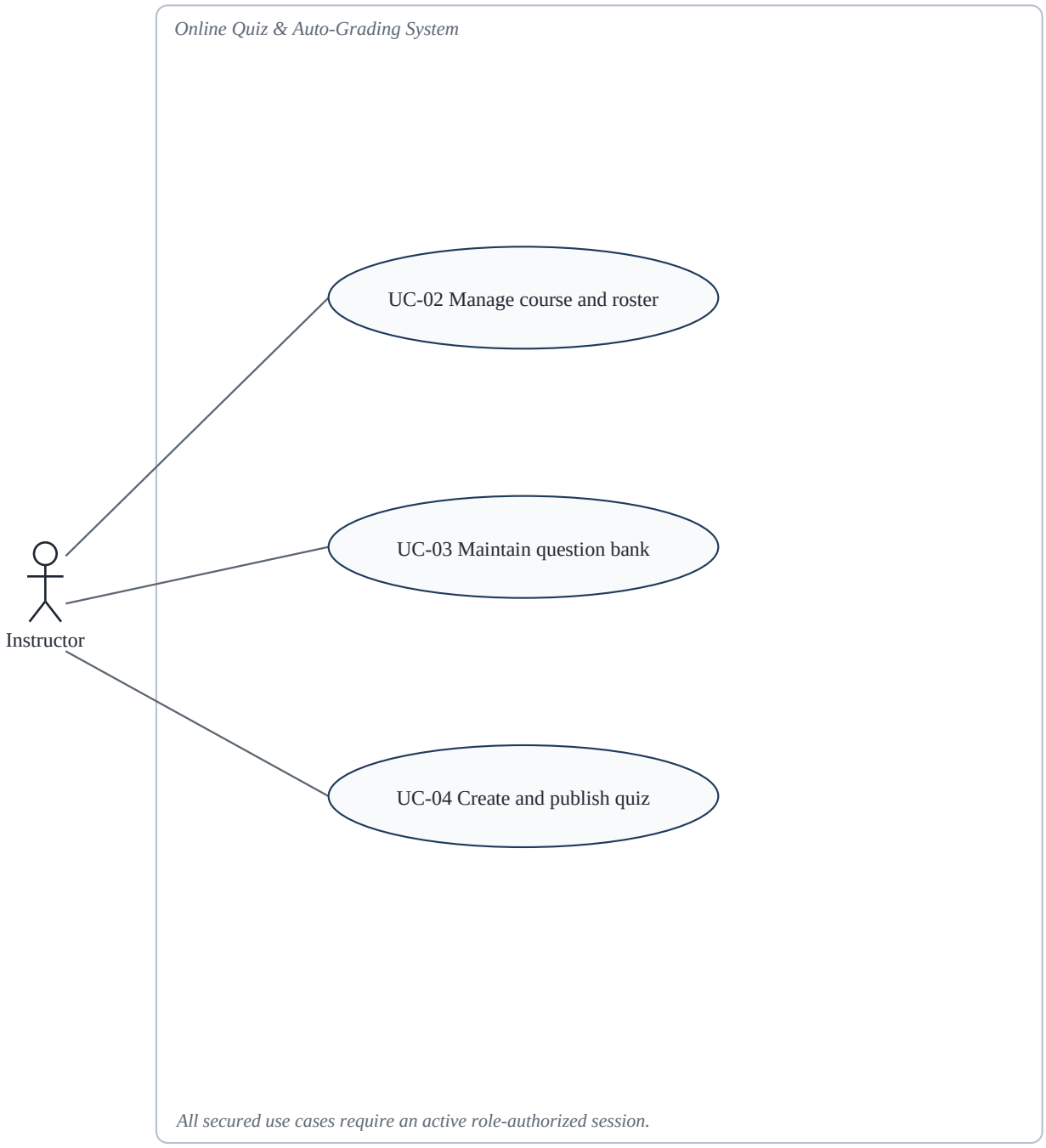


Figure 1 shows the instructor actions used to prepare a course, maintain its question bank, and publish a quiz.

3.2.1 Use Case Diagrams (continued)

Figure 2. Student quiz attempt

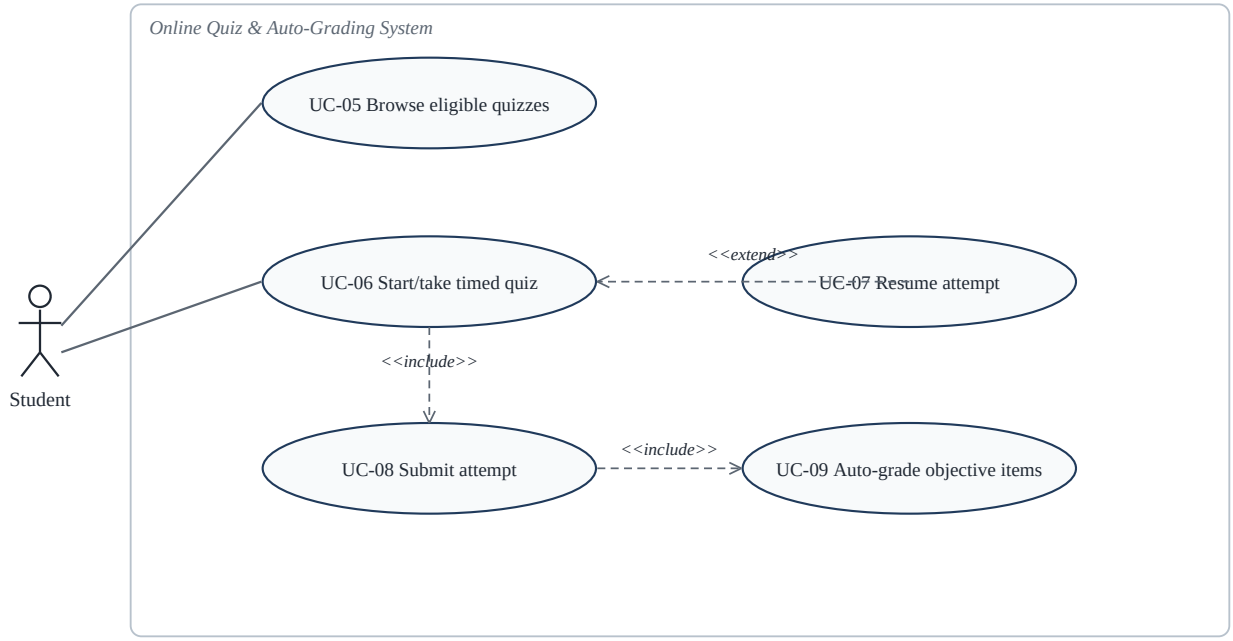
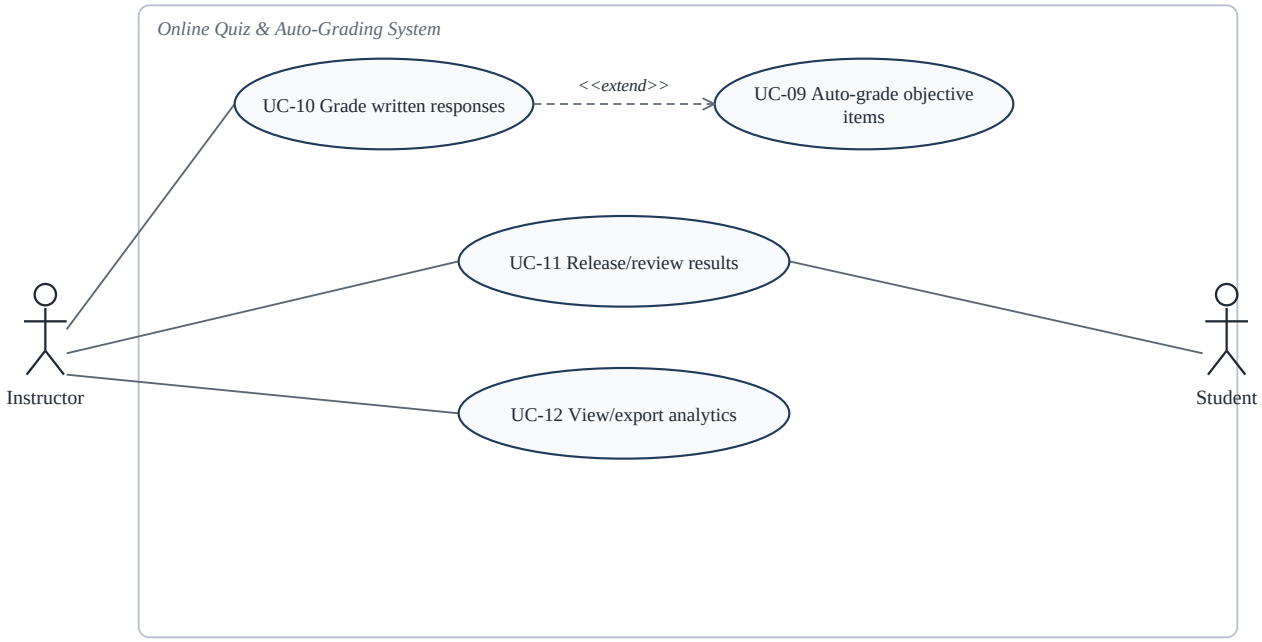


Figure 3. Grading, results, and reporting



Figures 2 and 3 show the student attempt lifecycle and the instructor grading, release, and reporting workflow.

3.2.2 Use Case Descriptions

UC-01 - Sign in

Primary actors: Student, Instructor, Administrator. **Preconditions:** The account exists and is active. **Trigger:** The user submits the sign-in form. **Main flow:** The system validates credentials, loads role permissions, starts a session, and opens the correct home page. **Alternatives:** Invalid credentials produce a generic error; inactive accounts are denied; a service interruption offers retry. **Postcondition:** Only a valid user has an active session.

UC-02 - Manage course and enrollments

Primary actor: Instructor or Administrator. **Preconditions:** The actor is authorized for course setup. **Trigger:** The actor creates a course or changes its roster. **Main flow:** The actor enters course details, adds registered student accounts, reviews the roster, and saves. The system checks required values and duplicate enrollment. **Alternatives:** Unknown account, duplicate code, or duplicate enrollment is rejected with an explanation. **Postcondition:** Course and roster changes are stored and attributable.

UC-03 - Maintain question bank

Primary actor: Instructor. **Preconditions:** The instructor owns or is assigned the course. **Trigger:** The instructor creates, edits, archives, or searches an item. **Main flow:** The instructor supplies prompt, type, points, and key or rubric; the system validates and saves the item version. **Alternatives:** Missing options, key, or rubric is highlighted; published-item edits create a safe new version. **Postcondition:** The valid item is available to authorized quiz authoring.

UC-04 - Create and publish quiz

Primary actor: Instructor. **Preconditions:** An active course and usable question bank exist. **Trigger:** The instructor selects New Quiz. **Main flow:** The instructor chooses items, sets order and points, configures availability, duration and attempts, previews, and publishes. **Alternatives:** The system blocks publication for an empty quiz, invalid window, missing key, or invalid scoring rule and identifies each issue. **Postcondition:** The published quiz is visible only to eligible students.

3.2.2 Use Case Descriptions (continued)

UC-05 - Browse eligible quizzes

Primary actor: Student. **Preconditions:** The student is signed in. **Trigger:** The student opens My Quizzes. **Main flow:** The system checks active enrollments and lists assigned quizzes with status, dates, duration, and attempts remaining. **Alternatives:** If no quiz is available, an explanatory empty state appears; closed or unauthorized quizzes cannot be opened. **Postcondition:** The student can select only a quiz allowed by course membership and schedule.

UC-06 - Start and take a timed quiz

Primary actor: Student. **Preconditions:** The quiz is open, the student is enrolled, and attempts remain. **Trigger:** The student selects Start. **Main flow:** The system rechecks eligibility, creates one attempt, starts the server timer, displays questions, and saves responses. **Alternatives:** A closed window, attempt limit, or existing active attempt prevents a second start; network interruption returns the student to the same attempt. **Postcondition:** One timed attempt is associated with the student and quiz.

UC-07 - Resume interrupted attempt

Primary actor: Student. **Preconditions:** The student owns an unsubmitted attempt. **Trigger:** The student returns to the active quiz. **Main flow:** The system restores the last server-confirmed responses, original question order, and remaining time, then allows work to continue if the deadline has not passed. **Alternatives:** If time expired, editing remains locked and the last saved answers are submitted; if no saved response exists, the item remains blank. **Postcondition:** The existing attempt continues or is finalized; no duplicate attempt is created.

UC-08 - Submit quiz attempt

Primary actor: Student; the system may submit at timeout. **Preconditions:** An active attempt exists. **Trigger:** The student confirms Submit or the server deadline arrives. **Main flow:** The system records final server-received answers, freezes editing, and confirms receipt. **Alternatives:** The student may return from the unanswered-item warning; repeated requests are idempotent; a lost acknowledgment permits status refresh without resubmitting a second attempt. **Postcondition:** The attempt is submitted once and awaits grading or release.

3.2.2 Use Case Descriptions (continued)

UC-09 - Auto-grade objective responses

Primary actor: System. **Preconditions:** A submitted attempt and published answer-key version exist. **Trigger:** Submission is accepted. **Main flow:** The system compares objective answers using the configured rule, assigns item scores, records the rule version, and calculates the objective subtotal. **Alternatives:** Unanswered items receive the configured default; malformed or missing key data is flagged for instructor review rather than silently scored. **Postcondition:** Reproducible objective marks are stored for the attempt.

UC-10 - Manually grade written responses

Primary actor: Instructor. **Preconditions:** The attempt is submitted and the instructor is authorized for its course. **Trigger:** The instructor opens pending written responses. **Main flow:** The instructor applies the rubric, enters a mark within the allowed range, and saves optional feedback. **Alternatives:** An out-of-range score is rejected; unsaved edits remain visibly pending; a changed mark creates an audit entry. **Postcondition:** Written marks are saved and the attempt total is recalculated.

UC-11 - Release and review results

Primary actors: Instructor and Student. **Preconditions:** Grading is complete or partial release is permitted. **Trigger:** The instructor releases results, or the student opens a released attempt. **Main flow:** The instructor selects release scope; the system records time and applies visibility rules. The student sees score and permitted feedback. **Alternatives:** Incomplete marks block full release; unreleased answers and private notes remain hidden. **Postcondition:** Result access matches the selected release policy.

UC-12 - View and export analytics

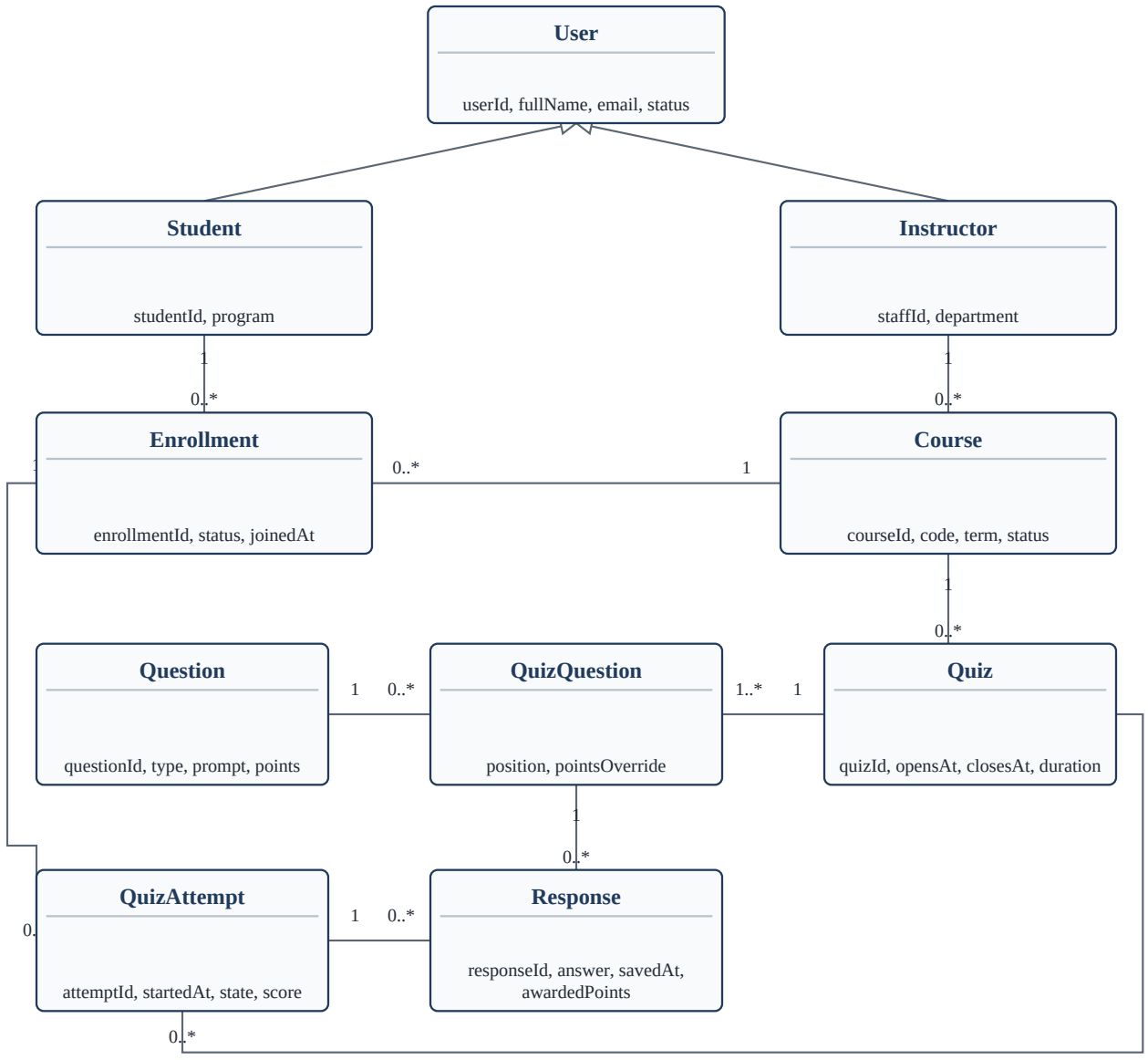
Primary actor: Instructor. **Preconditions:** The instructor is authorized for the selected course. **Trigger:** The instructor opens a quiz report and selects filters. **Main flow:** The system computes attempt count, score distribution, average, and question correctness; the instructor may export the filtered summary as CSV. **Alternatives:** No attempts produces an empty-state message; unauthorized course identifiers return no data. **Postcondition:** The report and exported rows reflect the same filters and permissions.

3.3 Domain Model

3.3.1 Conceptual Class Diagram

The model separates users, course membership, reusable questions, quiz-specific question placement, attempts, and responses. Multiplicities are shown at association ends. Quiz settings retain timing, release policy, and the answer-key version used for grading.

Figure 4. Conceptual class diagram with association multiplicities



An attempt belongs to one active enrollment and one quiz; each response is unique within its attempt and quiz-question placement.

3.3.2 Class Descriptions

User: Stores userId, fullName, email, and status; it is the parent of Student and Instructor. **Student:** Inherits User identity and participates in many courses through Enrollment. A student owns attempts through an active enrollment. **Instructor:** Inherits User identity and may own many Courses. **Course:** Stores courseId, code, term, and status; one instructor owns it, and it has many enrollments and quizzes. **Enrollment:** Stores enrollmentId, status, and joinedAt; each record links exactly one Student to one Course and may scope many attempts.

Question: Stores questionId, type, prompt, and points; one question may be placed in many quizzes through QuizQuestion. **Quiz:** Stores quizId, opensAt, closesAt, duration, limits, and release policy; it belongs to one Course and has many placements and attempts. **QuizQuestion:** Stores position and pointsOverride; it links exactly one Quiz and one Question. **QuizAttempt:** Stores attemptId, startedAt, state, and score; it belongs to one enrollment and one quiz and has many responses. **Response:** Stores responseId, answer, savedAt, awardedPoints, and optional feedback; it belongs to one attempt and one QuizQuestion, with at most one response for that pair.

4. Appendices

4.1 Appendix A: Data Dictionary

users - accounts, identity, status, and credentials. **user_roles** - role assignments for accounts. **courses** - course code, term, owner, and status. **enrollments** - student-course membership and state. **questions** - prompts, type, keys/rubrics, and version. **quizzes** - schedule, limits, and release policy. **quiz_questions** - question placement, order, and points per quiz. **quiz_attempts** - student attempts, timing, state, and total score. **responses** - answer, save time, item mark, and feedback. **audit_events** - security-sensitive changes such as role, roster, grade, and release actions.

The table names are conceptual; the final schema may normalize these entities while preserving the stated relationships and audit behavior.

3.4 Non-Functional Requirements

Each requirement below states a measurable target, a test method, and a pass criterion for later implementation evaluation.

NFR-01 - Performance

For a normal course load of up to 100 concurrent users, 95% of page and report requests shall complete within 2 seconds, excluding initial network transfer of large files. Quiz submission shall be acknowledged within 5 seconds under the same load. **Test:** Run a 10-minute load scenario with 100 virtual users and record server response times. **Pass:** At least 95% meet the limits and no accepted submission is lost.

NFR-02 - Security and access control

All protected operations shall enforce role and course permissions on the server; passwords shall never be stored as readable text; browser-server traffic shall use HTTPS in deployment. **Test:** Exercise a role/record access matrix, inspect stored account records, and attempt direct-object access as an unauthorized user. **Pass:** No unauthorized read or write succeeds, no plaintext password is present, and protected traffic is encrypted.

NFR-03 - Reliability and data integrity

A server restart or repeated submit request shall not create duplicate attempts or discard any response already confirmed as saved. Grading totals shall equal the sum of stored item marks. **Test:** Save responses, restart the service, retry submission, and compare attempt and score records. **Pass:** Each attempt is unique, all acknowledged answers remain, and the stored total matches the item marks.

NFR-04 - Usability and accessibility

A first-time student shall be able to locate an open quiz, answer a test item, and submit without staff assistance; quiz controls shall be keyboard operable and clearly indicate saved state, unanswered items, and remaining time. **Test:** Observe five representative users completing the task using keyboard and mouse. **Pass:** At least four complete the flow without help, and all controls can be reached and operated by keyboard.

NFR-05 - Browser and device compatibility

Core student and instructor workflows shall remain usable in the current stable versions of Chrome, Firefox, Safari, and Edge at desktop and mobile viewport widths. **Test:** Complete sign-in, quiz attempt, grading, and report review in each browser at 1280x800 and 390x844 viewports. **Pass:** No workflow is blocked, no text or control is clipped, and saved responses remain consistent across refresh.

3.5 External Interface Requirements

3.5.1 User Interface

The student interface shall show quiz status, instructions, question navigation, saved state, deadline, unanswered items, and a confirmed submission state. Instructor screens shall provide course, bank, quiz, grading, release, and report views. Forms shall identify required fields and explain validation errors in plain English.

3.5.2 Hardware Interface

No dedicated hardware is required. The system uses standard laptop, tablet, or phone displays and keyboard, mouse, or touch input. Camera, microphone, scanner, printer, and biometric devices are outside the Phase 2 scope.

3.5.3 Software Interface

The web application exchanges structured requests with its application service and relational database. The report function generates CSV files. Campus identity, learning-management, and external notification services are not required for the core version; any later integration must use documented interfaces and preserve the access rules in this SRS.

3.5.4 Communication Interface

Browser-to-server communication shall use HTTPS in deployment. Application requests and responses shall use a documented web API format; server-side timestamps govern quiz windows and deadlines. The client shall show pending or failed saves when connectivity is interrupted and shall confirm submission only after the server accepts it.

4.2 Appendix B: Glossary

Attempt: one recorded run of a quiz. **Auto-grading:** deterministic scoring from an answer key. **CSV:** comma-separated values file. **Enrollment:** a student's active membership in a course. **Feedback:** instructor-visible or student-visible comments associated with marks. **HTTP/HTTPS:** web communication protocol, with HTTPS providing encrypted transport. **Question bank:** a reusable collection of course questions. **RBAC:** permission checks based on assigned roles. **Release policy:** rule controlling when scores and feedback become visible. **SRS:** Software Requirements Specification. **Objective item:** item with a deterministic accepted response. **Subjective item:** item requiring instructor judgment.

End of Phase 2 SRS.